

Algorithm Design and Analysis Homework 5

Tianyi Guan 2400017423

2026 年 5 月 19 日

1. Problem 8.1

- (1) 因为需要判断补图，一个最简单的思想就是考虑两个输入的邻接矩阵对应元素加起来必须严格是每个元素都是 1 的 $n \times n$ 矩阵。在这个思路下，由于邻接矩阵是关于对角线对称的，因此只用遍历上三角矩阵，判断每个位置对应元素加起来是否为 1。伪代码参考：

Algorithm 1: 判定两个简单图是否为补图

Input: 两个简单图 G_1, G_2 的邻接矩阵 M_1, M_2 ，顶点数为 n

Output: 如果 G_1 是 G_2 的补图返回 True，否则返回 False

```
1 for  $i \leftarrow 1$  to  $n - 1$  do
2   for  $j \leftarrow i + 1$  to  $n$  do
3     if  $M_1[i][j] == M_2[i][j]$  then
4       return False;
5   end
6 end
7 end
8 return True;
```

- (2) 接下来证明第一小问的算法是严格下界。考虑存在某一种算法 A' ，其中复杂度低于 $\frac{n(n-1)}{2}$ ，那么其必然不能遍历顶点数为 n 的图的每一条边，进而我们可以在遗漏的那条边上，在 G_1 和 G_2 上都设置成 1 或都设置成 0，那么这个算法就无法判断 G_1 和 G_2 是否为补图了。

2. Problem 8.2

- (1) 算法的思想就是遍历每个项，维护一个变量 sum ，其中对每个项 i ，计算这一项的 $a_i * x^i$ 并加到 sum 上。但是连续求幂次的计算会浪费时间，因此采用逐步提取公因式的方法。伪代码如下：

Algorithm 2: 多项式求值算法

Input: 系数数组 $A = [a_0, a_1, \dots, a_n]$ ，自变量的值 x

Output: $P(x)$ 的计算结果

```
1  $sum \leftarrow a_n$ ;
   /* 从次高项一直遍历到常数项 */
2 for  $i \leftarrow n - 1$  to 0 step  $-1$  do
3    $sum \leftarrow sum \times x + a_i$ ;
4 end
5 return  $sum$ ;
```

- (2) 如果存在一个算法，其复杂度为 $o(n)$ ，那么其必然不能遍历多项式的全部 $n + 1$ 项。那么我们就可以随意改动没遍历到的这一项的系数，使得这个算法无法输出正确的多项式的值。因此所有多项式求值算法的复杂度都是 $\Omega(n)$ 。

3. Problem 8.4

- (1) 采用顺序归并的方式时，需要每次归并当前剩余未处理的数表的前两位，并把归并的结果放回未处理的数表的首位。因此，需要归并的总次数为 $\sum_{i=1}^{k-1} (i+1) * \frac{n}{k} = \frac{n(k+2)(k-1)}{2k}$ 次，也就是复杂度为 $O(nk)$ 。
- (2) 考虑对 k 个排好序的数表的队首元素维护一个堆，初始时对 k 个队首元素建堆，复杂度是 $O(k)$ ，随后开始归并过程，取出堆顶元素并将对应数表中的下一个元素放入堆中，这个过程每次需要 $O(\log k)$ ，一共 n 次。因此这个归并算法的复杂度一共是 $O(n \log k)$ 。
- (3) 采用**决策树模型**进行证明。将 k 个长度为 n/k 的已排序子表归并，等价于确定这 n 个元素在最终序列中的位置分配。由于各子表内部已有序，有效合并结果的总排列数（即决策树叶子节点数 N ）为：

$$N = \frac{n!}{((n/k)!)^k}$$

对于任意基于比较的算法，设其最坏情况比较次数为 h （决策树高度），则必须满足 $2^h \geq N$ ，即 $h \geq \log_2 N$ 。利用**斯特林近似公式** $\log_2(x!) \approx x \log_2 x - x \log_2 e$ 进行展开化简：

$$\begin{aligned} \log_2 N &= \log_2(n!) - k \cdot \log_2((n/k)!) \\ &\approx (n \log_2 n - n \log_2 e) - k \left(\frac{n}{k} \log_2 \left(\frac{n}{k} \right) - \frac{n}{k} \log_2 e \right) \\ &= n \log_2 n - n \log_2 e - n(\log_2 n - \log_2 k) + n \log_2 e \\ &= n \log_2 k \end{aligned}$$

因此，决策树的高度 $h \geq \Omega(n \log k)$ ，说明求解该问题的复杂度下界为 $\Omega(n \log k)$ 。

4. Problem 8.9

- (1) 只需要先对 L 中元素建最小堆，并且弹出 m 次堆顶元素就可以了。这个算法的复杂度是 $O(n) + m \cdot O(\log n) = O(n)$ 。
- (2) 任何一个解决这个问题的算法，就算不关注是不是正确的最小个 m 个数（认为这里可以常数时间取出来），至少需要确保这 m 个数是排好序的，否则我们可以调整 L 的内部顺序使得这 m 个数不按照正确顺序输出，因此这个算法必然不正确。而对这 m 个数进行排序的过程，就已经需要 $\omega(n/\log n \cdot \log(n/\log n)) = \omega(n)$ 的时间了，故必然不存在 $O(n)$ 的算法。

5. Problem 8.10

不妨设 $k < \frac{n+1}{2}$ ，否则可以调整为选第 $n+1-k$ 大的数，并且转变后续算法的不等号方向。

采用**对手论证 (Adversary Argument)**。设算法 A 在结束时找到了第 k 小的元素 x 。此时，算法在逻辑上必须建立一个大小为 $k-1$ 的较小元素集合 S ，以及一个大小为 $n-k$ 的较大元素集合 L 。为了确立这 n 个元素与 x 的相对大小关系，算法构建的比较关系图必须是连通的。将 n 个孤立节点连通至少需要构成一棵树，因此至少需要 $n-1$ 次基础比较。在上述构成连通图的基础比较中，若集合 S （共有 $k-1$ 个元素）仅通过单条路径连接至 x ，对手可以暗中将 S 中处于叶子节点的某个元素的值修改为极大值（即违背了它比 x 小的前提），由于比较关系树的脆弱性，算法 A 无法察觉此变动，从而给出错误结果。为了挫败对手的这种“作弊”行为，算法 A 必须确保 S 中的每一个元素都有**至少两条**比较路径，或者在 S 内部产生额外的比较环路，从而“锁死”它们小于 x 的事实。要为 $k-1$ 个元素建立额外的依赖关系，至少需要增加 $k-1$ 次额外的内部/交叉比较。综上所述，算法最少需要的比较次数为：

$$(n-1) + (k-1) = n + k - 2$$

因此，对于任意的 k ，选取第 k 小数所需的最少比较次数为 $n + \min\{k, n-k+1\} - 2$ 。证毕。